

Taleb Lean v0.2.3: independent audit and progress assessment

19 September 2026. Scope: the supplied v0.2.3 archive, the v0.2.2 verifier repairs it contains, the three new mathematics tasks, and the 158-family implementation ledger.

Assessment

Accept the mathematics milestone, with a remaining verifier correction, a regression-isolation issue, and two small documentation/ledger corrections. The new source supplies the three advertised mathematical slices. Independent compilation and axiom verification reproduce the reported 64 theorems, one instance and 16 aliases. I found no defect in the new formal propositions or their proofs within the reviewed scope.

The Gaussian result is the clearest increase in mathematical substance: actual probability measures now satisfy both premises of the conditional stable convolution theorem at $\alpha=2$. The independent-maximum identities are also useful probability results. The two-power-tail theorem correctly resolves the signed display at the formula level. None of these statements claims to complete the wider stable, EVT or sum-tail theories.

The broader ledger remains at **zero fully discharged families and ten marked partial**. This is a coarse family count, not a statement that no useful mathematics has been completed. All three scoped v0.2.3 tasks have delivered their specified slices, while their parent families contain additional obligations. The tracker should now record the Gaussian existence slice under T007 as well.

Independent reproduction

Check	Result
Archive SHA-256	Matches the supplied checksum; full value below.
Archive manifest	All 154 listed hashes match; 155 files total, with SHA256SUMS itself the sole unlisted file. No unsafe archive paths, duplicate member names or archived symlinks found.
Full patch	Applied v0.2.0 -> v0.2.3 to the original archive; all 155 output files reproduce the supplied v0.2.3 tree byte for byte.
Incremental patch	Reverse application reconstructs the v0.2.2 tree; forward reapplication reproduces v0.2.3 byte for byte. The intermediate ZIP's byte-level hash cannot be reproduced without that ZIP.
Clean project build	Exit 0, 26 project modules, 2736 Lake jobs, no compiler warnings or errors; 74.85 seconds in this environment. Dependency build artifacts came from the pinned Mathlib cache.
verify.py	Exit 0 / PASS; 146.47 seconds including its build and environment checks. All project source hashes match the received files.
Declaration and trust inventory	64 theorem declarations, 1 instance, 16 aliases: 81 public checks. All 158 project constants, including 41 internal constants, use only propext, Classical.choice and Quot.sound. No project axiom declarations.
Dependency state	Lean 4.24.0; all nine dependency revisions match the manifest and their working trees are clean.
Shipped regression suite	Sequential run: 6/10 match their intended gates, suite exit 1; 1627.10 seconds. Four later fixtures encounter an earlier private axiom. Fresh-copy reruns of those four cases and the dirty-dependency control all reach the correct rejection gate. See R1; a single clean 10/10 run was not reproduced.
Additional coverage probe	Unmodified verifier incorrectly accepts a nested orphan with a proof placeholder; recursive-discovery patch rejects it; healthy patched project passes. Details under V1.

Archive: 36155f3a2172d862a57c7a1d9bd4927f42c7b4b9da04c9332b52c1a094a23bf1.

Mathlib: f897ebcf72cd16f89ab4577d0c826cd14afaafc7. The trust scan imports 25 project modules; the separate verification module makes 26 build modules. These are consistent counts.

The supplied archive has no repository history. The claim that commit a5fe3002e7811f77ee4d01a6af6fc164fefe5b5b exists, that the original working tree is clean, and that nothing was pushed remains Fable's report. Patch/tree reproduction establishes the delivered contents independently.

Findings requiring follow-up

V1 - Medium: nested unimported Lean modules evade the completeness check. Location: scripts/verify.py, function modules_on_disk.

The new module-coverage check uses `glob('*.*lean')` in `AuditRepairs` and `Proofs`. It does not descend into subdirectories. In a separate copy, I added an unimported `AuditRepairs/Nested/Orphan.lean`

containing a private theorem of False proved with sorry. The unmodified v0.2.3 verifier returned exit 0 / PASS, retained the 158-constant inventory, and reported `all_disk_modules_imported=true`. Its source-hash list even records the added file, although that file was neither built nor scanned.

This does not invalidate the delivered proofs: the supplied project has no such nested source file, and its imported constants passed the axiom scan. It does invalidate the unconditional claim that every project module on disk is covered. This matters as the implementation grows into subdirectories.

The accompanying `recursive_module_coverage.patch` replaces the shallow walks with recursive walks and derives module names from their full relative paths. In independent runs, the nested fixture changes from exit 0 / PASS to exit 1 / FAIL naming `AuditRepairs.Nested.Orphan`, and the healthy project with the patch returns exit 0 / PASS. The existing ten-fixture suite was run on the received verifier; the new patch was tested with the nested negative case and the healthy positive control. Keep the current fail-closed public-API inventory policy when adding nested public declarations; recursive module discovery alone does not teach the intentionally narrow theorem regex every Lean syntax form.

R1 - Medium: the sequential ten-fixture regression does not reproduce as a clean acceptance run. Location: `scripts/harness_regression.py`, fixture restoration and reuse of the build tree.

The full independent run exits 1 after 1627.10 seconds, with 6/10 fixtures matching their intended gates. The protected-theorem, ordinary wrong-alias, optimized wrong-alias and flat-orphan cases all reject, but they reject because `audit_private_axiom` from an earlier fixture reappears in their scanned environments. Consequently, those runs do not establish the checks they were intended to test. The suite correctly marks these outcomes unexpected; it does not falsely declare itself passed.

I reran those four cases plus the dirty-dependency control in separate fresh copies, checking source hashes before applying each mutation. All five then fail at the correct gate. Alongside the first five successful cases in the sequential run, this establishes the ten intended fixture behaviours, but it is not a reproduction of a single clean 10/10 suite run. The original source snapshot was checked against the pristine baseline. The exact cause of the sequential contamination, including the role of source restoration, cached compiled modules or this managed execution environment, remains unresolved; these observations do not establish that Fable's own recorded run failed.

Use fresh fixture directories, or verify restored source hashes and rebuilt module isolation before each case. Preserve the negative fixture, exact failure reason and process exit status. A future release should include a clean full run with those controls and the new nested-module case. The raw sequential failure and all isolated reruns are retained in the evidence bundle. No additional placeholder or unapproved axiom was found in the received mathematical sources.

M1 - Low: the Gaussian documentation says "exactly when" / "iff", but the Lean theorem proves one direction. Location: `AuditRepairs/GaussianBridge.lean`, module description and `charFun_gaussianReal_eq_stableS1Expr` docstring.

The formal statement assumes $v = 2 \cdot \sigma^2$ and derives characteristic-function equality for every supplied t . That is precisely sufficient for this milestone. It does not prove the converse. Change the prose to "when", or add a separately quantified iff theorem if it is actually needed. A converse from equality at a single arbitrary t would be wrong: at $t=0$ all these characteristic functions equal one. No change to the current proof is needed.

L1 - Low: T007 understates its delivered special case. Location: docs/FORMALIZATION_BACKLOG.json and its Markdown companion, T007.

T007 is still labelled missing, although the new Gaussian law realizes S1 at $\alpha=2$. Record "partial: Gaussian existence established; general admissible S1 existence open" and cite the new bridge. Under that correction the tally becomes 11 partial / 91 missing, with every other count unchanged. This credits existing work; it is not a new theorem or a closed family. The companion ledger preserves the shipped status separately from this recommendation.

Review of the mathematics

Two-term power tails. `AuditTails.two_power_tail` covers ordered exponents with a nonnegative first weight and a positive weight on the heavier term. Its strict-index branch factors out the heavier power and proves the quotient tends to one; its equal-index branch combines the weights. `two_power_tail_min` handles either ordering with both weights positive. These are sound formula statements in the finite negative-log-ratio interface, including equal exponents and a zero lighter-term weight in the ordered theorem.

The statements allow arbitrary real exponents because they are analytic formulas. Probability survival functions need additional constraints and a law connection. The unrestricted signed-summand statement printed on page 99 remains false by cancellation; G18 now explains this correctly. The next nonnegative-sum target is a minimum finite logarithmic index. Event squeezing alone does not establish the stronger asymptotic tail equivalence suggested by an unqualified phrase such as "the heavier tail dominates".

Gaussian bridge. `varianceOfScale` enforces the variance $2\sigma^2$. The characteristic-function identity holds for any skew parameter β at $\alpha=2$. `gaussianParameters` supplies the admissible tuple, and `convolutionPower_gaussianReal_scale` actually applies `stableS1_convolutionPower_eq`; it does not silently take an unrelated direct Gaussian convolution proof. The variance form then uses $\sigma = \sqrt{v/2}$ and obtains $N(m, v)^{*n} = N(n*m, n*v)$.

Zero variance and $n=0$ are included, so the Dirac and zero-fold convolution cases are not excluded by hidden positive-scale or sample-size assumptions. This realizes the intended stable bridge at $\alpha=2$. It does not construct general $\alpha<2$ laws, identify an actual Cauchy law, or establish a stable domain-of-attraction theorem. Direct Gaussian convolution closure already exists in the pinned Mathlib; the project contribution here is its integration with the S1 interface.

Independent maxima. `measure_maxLeEvent` is a direct, appropriate use of `iIndepFun.meas_iInter`. It proves the product of coordinate threshold probabilities without an identical-distribution assumption. The shared-value lemmas give the power formula. The real-valued version explicitly assumes a finite measure and a nonempty finite index type; the latter supplies the positive cardinality required in the max-stability formulas. Strict/non-strict threshold choices are consistent and do not exclude atoms.

The basic event identity requires no additional measurability assumption because it uses the event-independence interface directly. It should not be read as a proof that arbitrary input functions are measurable. When packaging a pushforward-law theorem for a measurable maximum random variable, supply the corresponding measurability facts. The present Frechet/Gumbel results correctly assume the coordinate CDF identities at every point. Their existence, the reverse-Weibull family, minima and domains of attraction remain separate tasks.

The new source anchors agree with the supplied versioned book: printed page 99 / PDF 113 for the two-power display; printed page 140 / PDF 154 for Gaussian/stable scaling; printed page 173 / PDF 187 for the support-correct EVT formulas. The reviewed pre-existing public declaration types are unchanged from v0.2.1; the increment adds twelve theorem declarations and three definitions.

How far the wider implementation has progressed

Ledger state	Families	Interpretation
Fully discharged	0	No complete parent family is certified by this ledger.
Partial	10	Useful definitions or proofs exist; scope varies considerably.
Mathlib reuse candidates	4	Existing infrastructure must be instantiated; additional family-specific lemmas may remain.
Missing	92	Open implementation families, often with substantial dependencies.
Source check	34	Fix the exact statement before proof implementation; not all are established source errors.
Model needed	15	A mathematical model or decision problem is not yet fixed.
Empirical	3	Data reproduction and provenance tasks.
Total	158	Unequal obligation families, not atomic Lean theorems.

The ten supplied partial families are T001, T008, T029, T032, T046, T047, T060, T061, T118 and T124. Their exact delivered support and remaining boundaries are mapped in the companion ledger. In particular, T118 currently has a related pointwise convexity lemma, not a completed Pareto-moment convexity/Jensen theorem; T124 has corrected algebra, not the inverse-gamma expectation integral.

There are 140 families in the formal proof working set and 18 model/data families. The share marked partial is $10/158 = 6.3\%$, or $10/140 = 7.1\%$ of the proof working set. **Neither is a completion percentage.** Partial families are not uniformly half finished; some contain only formulas, while others now contain actual probability results. The total work remaining cannot be inferred from these ratios.

Release	Partial	Reuse	Missing	Source check	Model/data
v0.2.0	8	4	95	33	18
v0.2.1	8	4	94	34	18
v0.2.2	8	4	93	35	18
v0.2.3	10	4	92	34	18

In the mathematics increment, T029 moves from source-check to partial and T060 from missing to partial. T046 was already partial; its status stays the same while its substance increases significantly. This is why the status count alone understates the milestone.

The 81 public checks include 16 aliases, which are alternative entry points for existing results. The 64 theorem declarations include auxiliary results, variants and diagnostics. Neither number is a count of

completed book claims. The numerical coincidence between 158 scanned Lean constants and 158 backlog families has no mathematical meaning.

All 30 numbered and nine lettered units are indexed, with 353 bookmarks, 315 equation-locator occurrences and 71 statement-heading candidates. This is broad source coverage, not a complete atomic census of every unnumbered claim or calculation. The family register can grow as the formalization exposes further obligations. This audit reconciles every ledger row; it does not re-derive every chapter's mathematics.

Recommended next milestones

1. **Finish the verifier correction, isolate the regression fixtures, and align the two small documentation items.** Keep the existing ten regression fixtures and add an unimported nested-module fixture. Validate fresh or restored source hashes and build isolation, and preserve exact commands and exit statuses in every package. The provided small verifier patch is a concrete starting point.
2. **Close the generic max/min family T060.** Add the independent-minimum survival product and common-tail power formula, with a finite nonempty index and atom-safe threshold conventions. This work does not depend on constructing Frechet or Gumbel measures. It is the most direct candidate for the first fully closed ledger family.
3. **Construct the EVT measures, split into explicit child tasks under T061.** Gumbel first, Frechet second, then reverse-Weibull and the required location/scale wrappers. Prove support, monotonicity, right continuity, endpoint limits, mass one and CDF identification; then instantiate the maximum results with actual coordinate laws. Do not close T061 after only two of its three named families. The existing Stieltjes/CDF bridge is the appropriate infrastructure. [Mathlib CDF documentation](#).
4. **Finish T029's nonnegative-sum probability statement.** Prove positive threshold scaling and the finite-log-index squeeze, then the weighted finite-sum event inclusions. Independence is not required for this target. Keep exact convolution asymptotics in the separate subexponential workstream.
5. **Build an exact Pareto measure slice.** Reuse `paretoMeasure`; prove survival, finite moments, divergence at and beyond the exact-Pareto threshold, and threshold excess. Then connect the power pushforward. This advances T005/T021/T028/T032 and unlocks many later moment, estimator and payoff tasks, but does not complete every one of those broad families. General regularly varying tails need their own treatment at the equality boundary. [Mathlib Pareto documentation](#).
6. **Develop the longer foundations as separate workstreams.** Positive measurable regular variation, uniform convergence, Potter bounds and Karamata; concrete subexponential examples and closure; Cauchy and general stable existence; then domains of attraction and normalized-sum convergence. Build the kappa, Gini, estimator, mixture and application chapters on those explicit contracts. The existing dependency-group and chapter tables in the companion ledger keep all 158 families visible.

Track local formal verification, full family completion, and upstream Mathlib acceptance separately. This archive establishes no upstream merge. Gaussian closure and the independent-event product already reuse Mathlib infrastructure; new broadly useful Pareto, tail and distribution lemmas are more

natural candidates for extension. Check current upstream before proposing a contribution. Mathlib's current guidance also requires human understanding of AI-assisted code and disclosure of tool use. [Contribution guidance](#).

Evidence, limits and handoff

The companion Markdown register is readable directly; its JSON preserves all 158 source targets and assumptions, status history, declaration evidence, chapter/group rollups, and recommended acceptance boundaries. It deliberately leaves book-wide and effort-weighted completion percentages undefined. Future releases should add stable child-obligation IDs and count closed parent families only when all required child targets are met.

The evidence bundle includes the independent verification outputs, shipped-suite reproduction, additional nested-module probe, tested verifier patch, patch-replay result, and the scripts used for this audit. It excludes the multi-gigabyte toolchain and dependency caches. The received project is preserved without modification. Audit work copies and generated reports are separate.

No more files are needed for this mathematics assessment. The separate quantum/classical software and the paper's claim are not audited by this package. For that work the missing inputs are the actual software repository and revision, reproducible run command, input/output examples, and the precise paper version and claim to connect to the software.